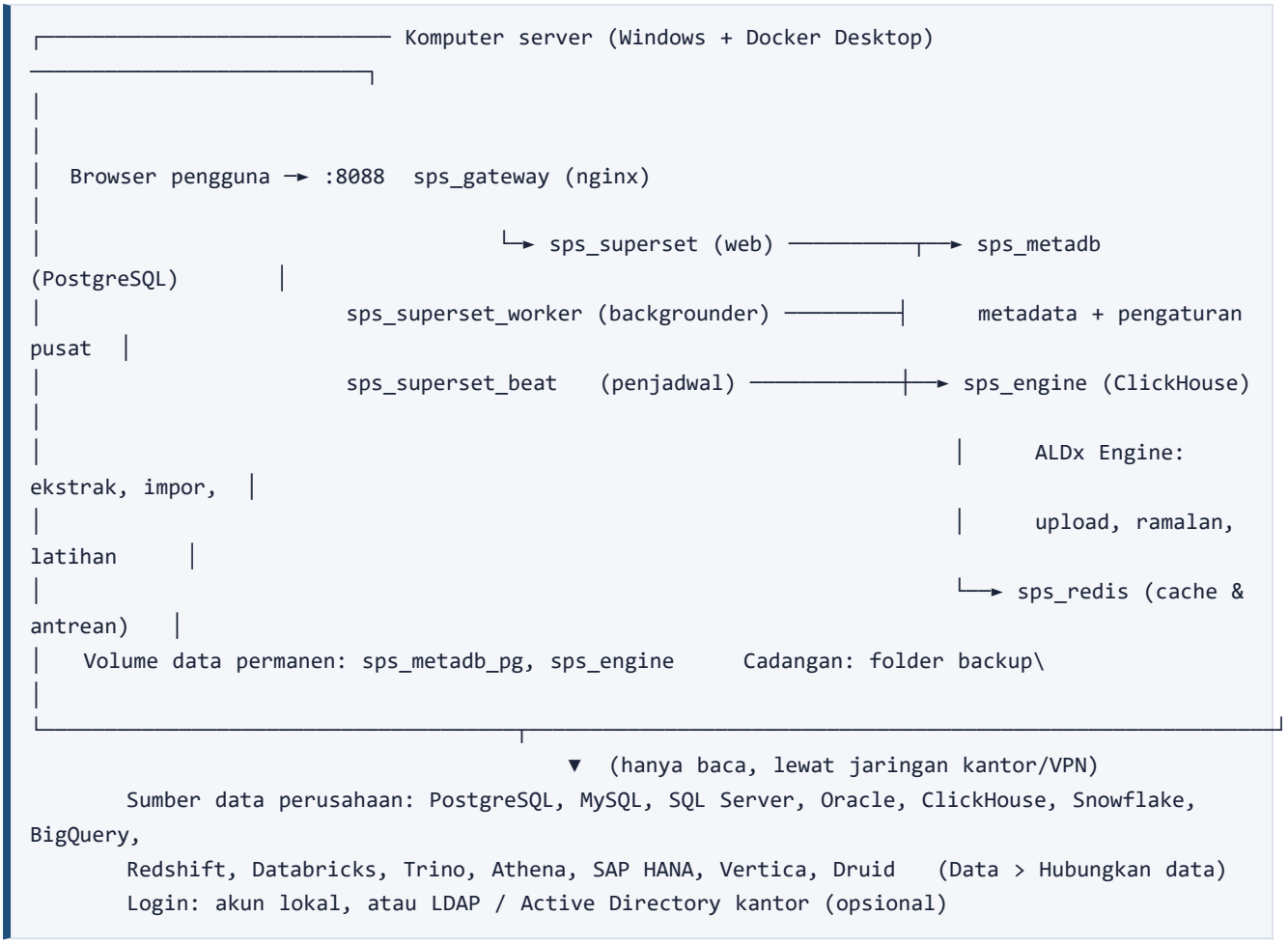
# 01 · Arsitektur

## Gambaran besar

Semua komponen berjalan di **Docker** (tidak lagi memakai XAMPP).




| Container | Fungsi | Data permanen? | Rincian |
|---|---|---|---|
| `gateway` | pintu masuk, pembagi beban | tidak | 11 |
| `superset` | aplikasi web (Superset + modul ALDx) | tidak | – |
| `superset-worker` | query async, ekstrak, laporan, ramalan | tidak | 11, 13 |
| `superset-beat` | penjadwal | tidak | – |
| `metadb` | PostgreSQL: user, chart, dashboard, koneksi, pengaturan, identitas | **ya** (volume `sps_metadb_pg`) | 14 |
| `engine` | ClickHouse: ALDx Engine | **ya** (volume `sps_engine`) | 13 |
| `redis` | cache & antrean | tidak | – |

Container tanpa data permanen boleh dihapus & dibuat ulang kapan saja. Data permanen ada di **volume Docker** dan dicadangkan dengan `scripts\cadangkan.ps1`.

## Modul ALDx di atas Superset ( `config/` )

| Modul | Halaman | Fungsi |
|---|---|---|
| `sps_identitas/` | login, semua halaman | nama, logo, warna, hiasan & animasi; halaman login beranimasi |
| `sps_security.py` , `sps_ldap.py` | login | akun lokal + LDAP/Active Directory (opsional) |
| `sps_akses.py` , `akses.yaml` | – | hak akses data otomatis dari profil pengguna (profil DJP; opsional) |
| `sps_koneksi/` | tab Hubungkan data | wizard koneksi database/data warehouse + impor Access/Excel/CSV |
| `sps_ai/` | `/asisten/` , panel AI | Asisten AI, peran ahli ( `peran.py` ), auto chart/dashboard, story (10, 23) |
| `sps_server/` | `/pusat-kendali/` | pengaturan server, auto-tuning, multi-node, batas data ( `batas.py` ) |
| `sps_engine/` | tab Ekstrak | ALDx Engine: ekstrak, upload, tugas terjadwal |
| `sps_app/` | `/app/` | cangkang aplikasi: Beranda, sidebar, palet perintah, bantuan, bahasa, tampilan ponsel, panel AI, ekspor (22) |
| `sps_superadmin.py` | – | role Superadmin di atas Admin & penjaganya (24) |
| `sps_kueri/` | `/kueri/` | Kueri Visual: susun kueri dengan klik, gabung tabel otomatis (21) |
| `sps_menu.py` | navbar | izin menu alur BI (dipakai sidebar cangkang `/app/` ) |
| `sps_pengguna.py` | installer | membuat akun lokal dari tabel pengguna installer |
| `sps_studio/` | `/studio/` | lembar kerja seret-lepas (23 chart, peta, ramalan), dashboard, persiapan data |
| `sps_latihan.py` , `sps_uji.py` | – | data latihan fiktif (penjualan ritel) & uji otomatis |
| `sps_pindah_meta.py` | – | memindahkan metadata versi lama (MySQL/MariaDB) ke PostgreSQL |
| `sps_dash_*.py` , `sps_connections.py` | – | dashboard sebagai kode & koneksi dari `.env` (profil DJP) |

## Istilah penting

| Istilah | Arti |
|---|---|
| **Metadata DB** | database tempat Superset menyimpan *dirinya sendiri*: user, role, dashboard, chart, koneksi. Di sini: `sps_superset` di container `metadb` |
| **Sumber data** | database milik perusahaan yang dianalisis. Hanya dibaca |
| **ALDx Engine** | database analitik internal (ClickHouse) untuk ekstrak, upload, ramalan |
| **Container** | "mesin mini" Docker. Bisa dibuat ulang tanpa kehilangan data karena data ada di volume |

| Istilah | Arti |
|---|---|
| **Volume** | penyimpanan permanen milik Docker ( `sps_metadb` , `sps_engine` ) |
| `host.docker.internal` | nama yang dipakai container untuk menunjuk komputer Windows (mis. Ollama) |

## Alur konfigurasi

```
.env ──► docker-compose.yml ──► environment container ──► superset_config.py ──► modul sps_*
                                                                  ▲
                        Pusat Kendali (tabel sps_server_atur) ────┘   batas, cache, timeout, proses
```

1. `.env` menyimpan alamat, akun, password, dan versi komponen.
2. `docker-compose.yml` mengirim isi `.env` ke container; di node utama `METADB_HOST=metadb` , `ENGINE_HOST=engine` .
3. `config/superset_config.py` membaca nilai itu ( `sps_server/lingkungan.py` ) dan pengaturan pusat dari metadb.

## Mengapa begini?

- **Docker**: Superset resmi tidak mendukung instalasi langsung di Windows; semua komponen satu cara pasang.
- **Redis**: cache bersama & antrean, syarat backgrounder dan multi-node.
- **Image sendiri ( `docker/Dockerfile` )**: image resmi + driver semua jenis database dari `docker/wheels/` (offline, tanpa internet).
- **Login LDAP via `ldap3`** : Python murni, tanpa kompilasi `python-ldap` (05).

## Pilihan database internal (disepakati)

Ada dua database milik aplikasi sendiri. Keduanya dipilih yang **teringan & tercepat** untuk tugasnya dan sudah terpasang otomatis di Docker.

### Metadata → PostgreSQL 17 (alpine)

| Kriteria | PostgreSQL ✓ | MariaDB/MySQL | SQLite |
|---|---|---|---|
| Rekomendasi resmi Superset | **ya** (dipakai di image & uji resmi) | didukung, butuh tambalan SEQUENCE | hanya untuk uji, tidak untuk produksi |
| Banyak pengguna & proses bersamaan | baik (MVCC) | baik | buruk (kunci berkas) |
| Pemakaian RAM saat diam | sangat kecil ( `shared_buffers` diatur auto-tuning) | kecil–sedang (InnoDB buffer) | – |
| Cadangan konsisten tanpa mematikan | `pg_dump` | `mariadb-dump` | salin berkas (harus berhenti) |
| Multi-node (node tambahan) | ya | ya | tidak |

Versi lama (MySQL XAMPP / MariaDB) dipindah otomatis oleh `PASANG.cmd` (`scripts\pindah-metadata.ps1`).

## ALDx Engine → ClickHouse (profil ringan)

| Kriteria | ClickHouse ✓ | DuckDB | PostgreSQL |
|---|---|---|---|
| Kecepatan agregasi jutaan baris | sangat cepat (kolumnar, paralel) | sangat cepat | lambat untuk analitik besar |
| Banyak pengguna membaca **sambil** ekstrak menulis | ya (server) | **tidak** (satu penulis, berkas terkunci) | ya |
| Dipakai dari beberapa node web/worker | ya (lewat jaringan) | tidak (berkas lokal) | ya |
| Kompresi data | 5–10× | 3–5× | 1–2× |
| Pemakaian RAM | diatur auto-tuning; profil ringan mematikan log sistem tak terpakai & mengecilkan cache | sangat kecil | sedang |

DuckDB tetap dipakai sebagai **format berkas** yang bisa diimpor (Hubungkan data → Berkas), bukan sebagai engine server, karena tidak aman untuk banyak penulis/pembaca bersamaan.